Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 3
з дисципліни: «Теорія алгоритмів»
Тема: «Спеціалізована структура даних Heap як основа ефективного нестійкого сортування»

Варіант № 22

Виконав:
Студент групи AI-254
Церковний О. Г.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритму сортування купою (Heapsort). За рахунок виконання трасування та візуалізації у вигляді графа студент має практично довести логарифмічну часову складність та нестійкість алгоритму сортування купою, а також порівняти цей алгоритм за кількістю базових операцій з алгоритмом швидкого сортування.

Завдання:
1. Спроєктувати, реалізувати, протестувати та проаналізувати алгоритм сортування купою.
2. Виконати трасування та візуалізацію двох фаз алгоритму сортування купою, надати пояснення.
3. Довести на прикладі, що алгоритм сортування купою є нестійким, та порівняти обчислювальну складність алгоритмів сортування купою та швидкого сортування за базовими операціями.

Варіант	Послідовність
22	19, 75, 43, 31, 5, 66, 62, 34, 76

Результати виконання завдання:

1. Сортування купою (HeapSort)

Для реалізації алгоритму сортування купою використовується структура даних максимальна купа (Max-Heap), у якій значення кожного батьківського вузла більше або дорівнює значенням його дочірніх вузлів, тобто $A[i] \geq A[2i + 1]$ та $A[i] \geq A[2i + 2]$. Завдяки цьому найбільший елемент завжди знаходиться в корені купи $A[0]$.

Алгоритм складається з двох основних фаз:

1) Побудова максимальної купи – перетворення початкового масиву A = [19, 75, 43, 31, 5, 66, 62, 34, 76] (n = 9) на структуру Max-Heap за допомогою процедури занурення Sink для вузлів з індексами i = 3, 2, 1, 0.

2) Сортування – послідовний обмін кореня купи з останнім елементом, зменшення розміру купи на 1 та відновлення її властивостей процедурою Sink.

Псевдокод алгоритму Heap_Max та допоміжної процедури Sink наведено в таблиці 3.1, а псевдокод алгоритму HeapSort, який складається з фаз побудови максимальної купи та сортування, – в таблиці 3.2.

Таблиця 3.1 – Приклад псевдокоду алгоритму створення максимальної купи на базі процедури занурення

```
Heap_Max(A)
// Вхідні дані: невідсортований масив A
// Вихідні дані: масив A, який є максимальною купою
   n ← |A|
   // Починаємо з першого елемента, що має дочірні вузли, та рухаємося до
початку масиву
   for i ← floor(n / 2) - 1 down to 0 do
      Sink(A, i, n)
```

```
Sink(A, i, n)
// Вхідні дані: масив A, індекс кореня піддерева i, n - розмір купи
// Вихідні дані: масив A, де елемент A[i] занурився на потрібне місце
   k ← i
   while True do
      j ← 2 * k + 1  // Індекс лівого дочірнього елемента

      // Перевіряємо, чи існує лівий дочірній елемент
      if j >= n then
         break

      // Знаходимо індекс найбільшого дочірнього елемента
      if j + 1 < n and A[j + 1] > A[j] then
         j ← j + 1

      // Порівнюємо поточний елемент з найбільшим дочірнім
      if A[k] >= A[j] then
         break  // Елемент на своєму місці, вихід з циклу
      else
         Swap(A[k], A[j])
         k ← j
```

Таблиця 3.2 – Приклад псевдокоду алгоритму сортування купою на базі процедури занурення Sink(A, i, n)

```
HeapSort(A)
// Вхідні дані: невідсортований масив A
// Вихідні дані: відсортований масив A
   n ← |A|
   // Фаза 1: Побудова максимальної купи
   for i ← floor(n / 2) - 1 down to 0 do
      Sink(A, i, n)

   // Фаза 2: Сортування. Повторно вилучаємо максимальний елемент і переміщуємо
його в кінець масиву
   for i ← n - 1 down to 1 do
      Swap(A[0], A[i])
      // Відновлюємо властивості купи для зменшеного масиву
      Sink(A, 0, i)
```

Для практичної реалізації алгоритму було розроблено програму мовою Python, яка виконує побудову максимальної купи та подальше впорядкування її елементів для варіанта 22. Програма також підраховує базові операції (порівняння значень елементів та присвоювання, один обмін – 3 присвоювання; методика підрахунку така сама, як у ЛР1: враховуються і службові присвоювання змінним k, j, n) для подальшого порівняння з алгоритмом швидкого сортування. Відповідний програмний код наведено в Лістингу 3.1.

Лістинг 3.1 – Python-код реалізації алгоритму сортування купою з підрахунком базових операцій

```
def swap(arr, i, j):
    """Міняє місцями два елементи в масиві."""
    arr[i], arr[j] = arr[j], arr[i]
    return 3  # Кількість присвоювань при обміні

def sink(arr, i, n):
    """Процедура занурення елемента вниз по купі."""
    comparisons = 0
    assignments = 0
    k = i
    assignments += 1  # присвоєння k
    while True:
        j = 2 * k + 1  # Індекс лівого дочірнього елемента
        assignments += 1  # присвоєння j
        if j >= n:
            break
        # Знаходимо індекс найбільшого дочірнього елемента
        if j + 1 < n:
            comparisons += 1
            if arr[j + 1] > arr[j]:
                j += 1
                assignments += 1  # присвоєння j
        # Перевіряємо властивість максимальної купи
        comparisons += 1
        if arr[k] >= arr[j]:
            break
        # Міняємо місцями та продовжуємо занурення
        assignments += swap(arr, k, j)
        k = j
        assignments += 1  # присвоєння k
    return comparisons, assignments

def heapsort(arr):
    """Алгоритм сортування купою."""
    n = len(arr)
    comparisons = 0
    assignments = 0
    print(f"Початковий масив: {arr}\n")
    # Фаза 1: Побудова максимальної купи
    print("--- Фаза 1: Побудова максимальної купи ---")
    for i in range(n // 2 - 1, -1, -1):
        print(f"Занурюємо елемент з індексу {i}: {arr[i]}")
        c, a = sink(arr, i, n)
        comparisons += c
        assignments += a
```

```
    print(f"\nМасив після побудови купи: {arr}\n")
    # Фаза 2: Сортування
    print("--- Фаза 2: Сортування ---")
    for i in range(n - 1, 0, -1):
        print(f"Міняємо місцями корінь ({arr[0]}) та останній елемент
({arr[i]})")
        assignments += swap(arr, 0, i)
        n -= 1
        assignments += 1  # присвоєння n
        print(f"Розмір купи зменшився до {n}. Відновлюємо властивості купи.")
        c, a = sink(arr, 0, n)
        comparisons += c
        assignments += a
        print(f"Масив на поточному кроці: {arr}\n")
    return arr, comparisons, assignments

# Моделювання
A = [19, 75, 43, 31, 5, 66, 62, 34, 76]
sorted_A, comparisons, assignments = heapsort(A)
print(f"Відсортований масив: {sorted_A}")
print(f"Загальна кількість порівнянь: {comparisons}")
print(f"Загальна кількість присвоювань: {assignments}")
```

Результати виконання Python-коду, наведеного в Лістингу 3.1, представлено в таблиці 3.3.

Таблиця 3.3 – Результати виконання алгоритму сортування купою

Початковий масив: [19, 75, 43, 31, 5, 66, 62, 34, 76] --- Фаза 1: Побудова максимальної купи --- Занурюємо елемент з індексу 3: 31 Занурюємо елемент з індексу 2: 43 Занурюємо елемент з індексу 1: 75 Занурюємо елемент з індексу 0: 19 Масив після побудови купи: [76, 75, 66, 34, 5, 43, 62, 19, 31] --- Фаза 2: Сортування --- Міняємо місцями корінь (76) та останній елемент (31) Розмір купи зменшився до 8. Відновлюємо властивості купи. Масив на поточному кроці: [75, 34, 66, 31, 5, 43, 62, 19, 76] Міняємо місцями корінь (75) та останній елемент (19) Розмір купи зменшився до 7. Відновлюємо властивості купи. Масив на поточному кроці: [66, 34, 62, 31, 5, 43, 19, 75, 76]

```
Міняємо місцями корінь (66) та останній елемент (19)
Розмір купи зменшився до 6. Відновлюємо властивості купи.
Масив на поточному кроці: [62, 34, 43, 31, 5, 19, 66, 75, 76]

Міняємо місцями корінь (62) та останній елемент (19)
Розмір купи зменшився до 5. Відновлюємо властивості купи.
Масив на поточному кроці: [43, 34, 19, 31, 5, 62, 66, 75, 76]

Міняємо місцями корінь (43) та останній елемент (5)
Розмір купи зменшився до 4. Відновлюємо властивості купи.
Масив на поточному кроці: [34, 31, 19, 5, 43, 62, 66, 75, 76]

Міняємо місцями корінь (34) та останній елемент (5)
Розмір купи зменшився до 3. Відновлюємо властивості купи.
Масив на поточному кроці: [31, 5, 19, 34, 43, 62, 66, 75, 76]

Міняємо місцями корінь (31) та останній елемент (19)
Розмір купи зменшився до 2. Відновлюємо властивості купи.
Масив на поточному кроці: [19, 5, 31, 34, 43, 62, 66, 75, 76]

Міняємо місцями корінь (19) та останній елемент (5)
Розмір купи зменшився до 1. Відновлюємо властивості купи.
Масив на поточному кроці: [5, 19, 31, 34, 43, 62, 66, 75, 76]

Відсортований масив: [5, 19, 31, 34, 43, 62, 66, 75, 76]
Загальна кількість порівнянь: 34
Загальна кількість присвоювань: 141
```

2. Трасування та візуалізація алгоритму сортування купою

Для детального дослідження роботи алгоритму сортування купою програмний код Лістингу 3.1 було доповнено командами виведення проміжних результатів (Лістинг 3.2). Це дозволяє простежити вибір дочірнього елемента, порівняння та обміни під час побудови максимальної купи й подальшого сортування масиву для варіанта 22. Результати трасування обох фаз алгоритму HeapSort представлено в таблиці 3.4.

Лістинг 3.2 – Python-код реалізації алгоритму сортування купою (трасування)

```
def swap(arr, i, j):
    """Міняє місцями два елементи в масиві."""
    arr[i], arr[j] = arr[j], arr[i]

def sink(arr, i, n):
    """Процедура занурення елемента вниз по купі (з трасуванням)."""
    k = i
    print(f"Починаємо 'занурювати' елемент: {arr[k]} з індексу {k}")
    while True:
        j = 2 * k + 1  # Індекс лівого дочірнього елемента
        if j >= n:
            print(f"Елемент {arr[k]} на індексі {k} досяг кінця купи.
Завершуємо.")
            break
        print(f"  Обираємо лівий дочірній елемент: {arr[j]} на індексі {j}")
        # Знаходимо індекс найбільшого дочірнього елемента
        if j + 1 < n:
            if arr[j + 1] > arr[j]:
                print(f"  Порівнюємо {arr[j]} (лівий) та {arr[j + 1]} (правий).
Обираємо {arr[j + 1]}")
                j += 1
            else:
                print(f"  Порівнюємо {arr[j]} (лівий) та {arr[j + 1]} (правий).
Обираємо {arr[j]}")
        # Перевіряємо властивість максимальної купи
        if arr[k] >= arr[j]:
            print(f"  {arr[k]} (батько) >= {arr[j]} (найбільший дочірній).
Елемент на своєму місці.")
            break
        print(f"  Міняємо місцями {arr[k]} (батько) та {arr[j]} (дочірній)")
        swap(arr, k, j)
        print(f"  Масив після обміну: {arr}")
        k = j

def heapsort(arr):
    """Алгоритм сортування купою (з трасуванням)."""
    n = len(arr)
    print(f"Початковий масив: {arr}")
    # Фаза 1: Побудова максимальної купи
    print("--- Фаза 1: Побудова максимальної купи ---\n")
    for i in range(n // 2 - 1, -1, -1):
        sink(arr, i, n)
        print("----------------------------------")
    print(f"\nМасив після побудови купи: {arr}\n")
    # Фаза 2: Сортування
    print("--- Фаза 2: Сортування ---\n")
    for i in range(n - 1, 0, -1):
        print(f"Міняємо місцями корінь ({arr[0]}) та останній елемент
({arr[i]})")
        swap(arr, 0, i)
        n -= 1
        print(f"Розмір купи зменшився до {n}. Відновлюємо властивості купи.")
        sink(arr, 0, n)
        print(f"Масив на поточному кроці: {arr}\n")
    return arr

# Моделювання
A = [19, 75, 43, 31, 5, 66, 62, 34, 76]
sorted_A = heapsort(A)
print(f"Відсортований масив: {sorted_A}")
```

Таблиця 3.4 – Результати трасування алгоритму сортування купою

Початковий масив: [19, 75, 43, 31, 5, 66, 62, 34, 76]
--- Фаза 1: Побудова максимальної купи ---

Починаємо 'занурювати' елемент: 31 з індексу 3
  Обираємо лівий дочірній елемент: 34 на індексі 7
  Порівнюємо 34 (лівий) та 76 (правий). Обираємо 76
  Міняємо місцями 31 (батько) та 76 (дочірній)
  Масив після обміну: [19, 75, 43, 76, 5, 66, 62, 34, 31]
Елемент 31 на індексі 8 досяг кінця купи. Завершуємо.

Починаємо 'занурювати' елемент: 43 з індексу 2
  Обираємо лівий дочірній елемент: 66 на індексі 5
  Порівнюємо 66 (лівий) та 62 (правий). Обираємо 66
  Міняємо місцями 43 (батько) та 66 (дочірній)
  Масив після обміну: [19, 75, 66, 76, 5, 43, 62, 34, 31]
Елемент 43 на індексі 5 досяг кінця купи. Завершуємо.

Починаємо 'занурювати' елемент: 75 з індексу 1
  Обираємо лівий дочірній елемент: 76 на індексі 3
  Порівнюємо 76 (лівий) та 5 (правий). Обираємо 76
  Міняємо місцями 75 (батько) та 76 (дочірній)
  Масив після обміну: [19, 76, 66, 75, 5, 43, 62, 34, 31]
  Обираємо лівий дочірній елемент: 34 на індексі 7
  Порівнюємо 34 (лівий) та 31 (правий). Обираємо 34
  75 (батько) >= 34 (найбільший дочірній). Елемент на своєму місці.

```
Починаємо 'занурювати' елемент: 19 з індексу 0
  Обираємо лівий дочірній елемент: 76 на індексі 1
  Порівнюємо 76 (лівий) та 66 (правий). Обираємо 76
  Міняємо місцями 19 (батько) та 76 (дочірній)
  Масив після обміну: [76, 19, 66, 75, 5, 43, 62, 34, 31]
  Обираємо лівий дочірній елемент: 75 на індексі 3
  Порівнюємо 75 (лівий) та 5 (правий). Обираємо 75
  Міняємо місцями 19 (батько) та 75 (дочірній)
  Масив після обміну: [76, 75, 66, 19, 5, 43, 62, 34, 31]
  Обираємо лівий дочірній елемент: 34 на індексі 7
  Порівнюємо 34 (лівий) та 31 (правий). Обираємо 34
  Міняємо місцями 19 (батько) та 34 (дочірній)
  Масив після обміну: [76, 75, 66, 34, 5, 43, 62, 19, 31]
Елемент 19 на індексі 7 досяг кінця купи. Завершуємо.
--------------------------------

Масив після побудови купи: [76, 75, 66, 34, 5, 43, 62, 19, 31]

--- Фаза 2: Сортування ---

Міняємо місцями корінь (76) та останній елемент (31)
Розмір купи зменшився до 8. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 31 з індексу 0
  Обираємо лівий дочірній елемент: 75 на індексі 1
  Порівнюємо 75 (лівий) та 66 (правий). Обираємо 75
  Міняємо місцями 31 (батько) та 75 (дочірній)
  Масив після обміну: [75, 31, 66, 34, 5, 43, 62, 19, 76]
  Обираємо лівий дочірній елемент: 34 на індексі 3
  Порівнюємо 34 (лівий) та 5 (правий). Обираємо 34
  Міняємо місцями 31 (батько) та 34 (дочірній)
```

```
  Масив після обміну: [75, 34, 66, 31, 5, 43, 62, 19, 76]
  Обираємо лівий дочірній елемент: 19 на індексі 7
  31 (батько) >= 19 (найбільший дочірній). Елемент на своєму місці.
Масив на поточному кроці: [75, 34, 66, 31, 5, 43, 62, 19, 76]

Міняємо місцями корінь (75) та останній елемент (19)
Розмір купи зменшився до 7. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Обираємо лівий дочірній елемент: 34 на індексі 1
  Порівнюємо 34 (лівий) та 66 (правий). Обираємо 66
  Міняємо місцями 19 (батько) та 66 (дочірній)
  Масив після обміну: [66, 34, 19, 31, 5, 43, 62, 75, 76]
  Обираємо лівий дочірній елемент: 43 на індексі 5
  Порівнюємо 43 (лівий) та 62 (правий). Обираємо 62
  Міняємо місцями 19 (батько) та 62 (дочірній)
  Масив після обміну: [66, 34, 62, 31, 5, 43, 19, 75, 76]
Елемент 19 на індексі 6 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [66, 34, 62, 31, 5, 43, 19, 75, 76]

Міняємо місцями корінь (66) та останній елемент (19)
Розмір купи зменшився до 6. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Обираємо лівий дочірній елемент: 34 на індексі 1
  Порівнюємо 34 (лівий) та 62 (правий). Обираємо 62
  Міняємо місцями 19 (батько) та 62 (дочірній)
  Масив після обміну: [62, 34, 19, 31, 5, 43, 66, 75, 76]
  Обираємо лівий дочірній елемент: 43 на індексі 5
  Міняємо місцями 19 (батько) та 43 (дочірній)
  Масив після обміну: [62, 34, 43, 31, 5, 19, 66, 75, 76]
Елемент 19 на індексі 5 досяг кінця купи. Завершуємо.
```

Масив на поточному кроці: [62, 34, 43, 31, 5, 19, 66, 75, 76]

Міняємо місцями корінь (62) та останній елемент (19)
Розмір купи зменшився до 5. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Обираємо лівий дочірній елемент: 34 на індексі 1
  Порівнюємо 34 (лівий) та 43 (правий). Обираємо 43
  Міняємо місцями 19 (батько) та 43 (дочірній)
  Масив після обміну: [43, 34, 19, 31, 5, 62, 66, 75, 76]
Елемент 19 на індексі 2 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [43, 34, 19, 31, 5, 62, 66, 75, 76]

Міняємо місцями корінь (43) та останній елемент (5)
Розмір купи зменшився до 4. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 5 з індексу 0
  Обираємо лівий дочірній елемент: 34 на індексі 1
  Порівнюємо 34 (лівий) та 19 (правий). Обираємо 34
  Міняємо місцями 5 (батько) та 34 (дочірній)
  Масив після обміну: [34, 5, 19, 31, 43, 62, 66, 75, 76]
  Обираємо лівий дочірній елемент: 31 на індексі 3
  Міняємо місцями 5 (батько) та 31 (дочірній)
  Масив після обміну: [34, 31, 19, 5, 43, 62, 66, 75, 76]
Елемент 5 на індексі 3 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [34, 31, 19, 5, 43, 62, 66, 75, 76]

Міняємо місцями корінь (34) та останній елемент (5)
Розмір купи зменшився до 3. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 5 з індексу 0
  Обираємо лівий дочірній елемент: 31 на індексі 1
  Порівнюємо 31 (лівий) та 19 (правий). Обираємо 31

```
  Міняємо місцями 5 (батько) та 31 (дочірній)
  Масив після обміну: [31, 5, 19, 34, 43, 62, 66, 75, 76]
Елемент 5 на індексі 1 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [31, 5, 19, 34, 43, 62, 66, 75, 76]

Міняємо місцями корінь (31) та останній елемент (19)
Розмір купи зменшився до 2. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 19 з індексу 0
  Обираємо лівий дочірній елемент: 5 на індексі 1
  19 (батько) >= 5 (найбільший дочірній). Елемент на своєму місці.
Масив на поточному кроці: [19, 5, 31, 34, 43, 62, 66, 75, 76]

Міняємо місцями корінь (19) та останній елемент (5)
Розмір купи зменшився до 1. Відновлюємо властивості купи.
Починаємо 'занурювати' елемент: 5 з індексу 0
Елемент 5 на індексі 0 досяг кінця купи. Завершуємо.
Масив на поточному кроці: [5, 19, 31, 34, 43, 62, 66, 75, 76]

Відсортований масив: [5, 19, 31, 34, 43, 62, 66, 75, 76]
```

Для наочного представлення роботи алгоритму сортування купою на основі результатів трасування (таблиця 3.4) побудовано графічні схеми обох фаз. На рисунку 3.1 показано процес формування максимальної купи, а на рисунку 3.2 – послідовне переміщення найбільших елементів у відсортовану частину масиву. Червоні лінії на рисунках позначають шлях занурення елемента, а білі блоки – елементи, що вже стали на свої остаточні місця.

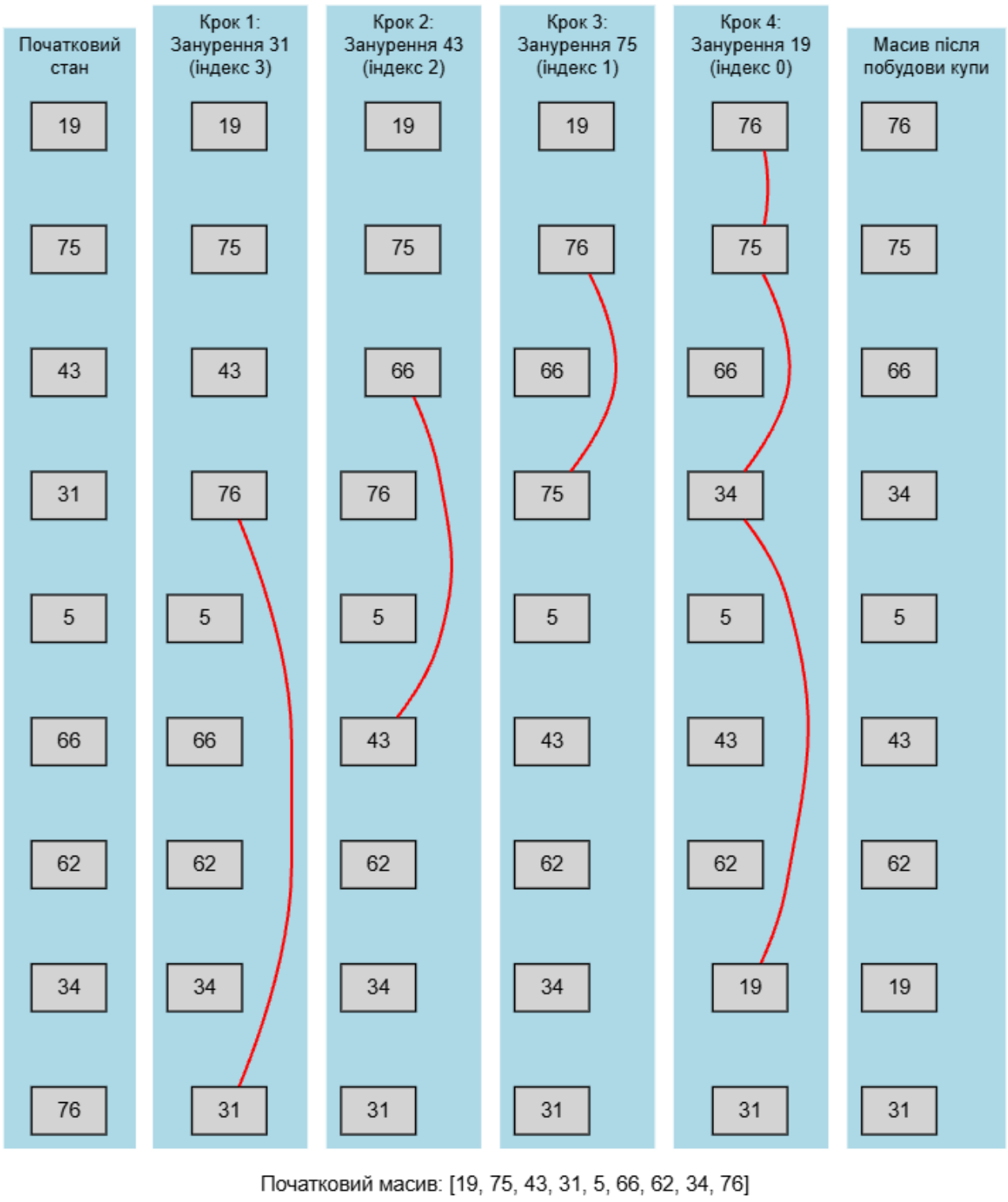


Рисунок 3.1 – Візуалізація кроків фази побудови максимальної купи алгоритму heapsort

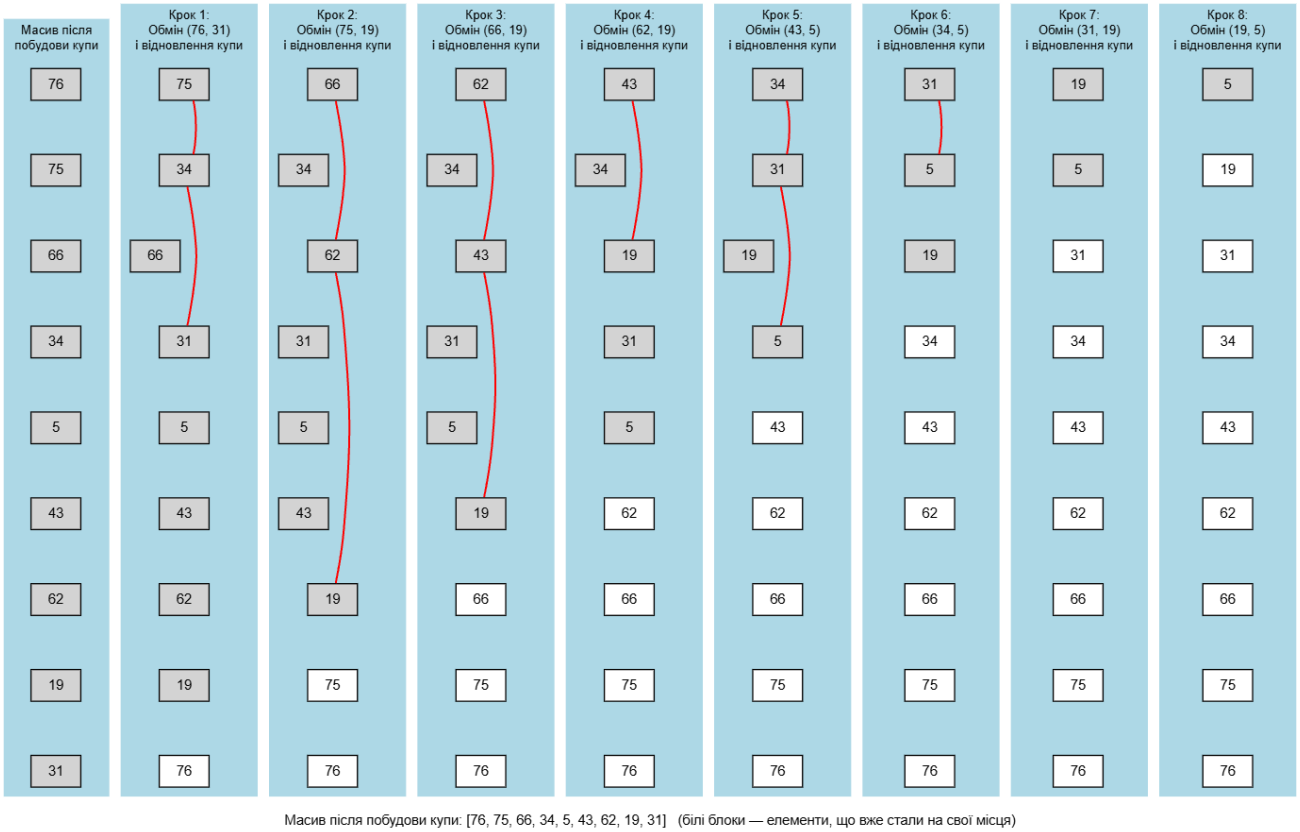


Рисунок 3.2 – Візуалізація кроків фази сортування алгоритму сортування heapsort

Пояснення до трасування та візуалізації фази 1 (побудова максимальної купи), рисунок 3.1

Вихідний масив A = [19, 75, 43, 31, 5, 66, 62, 34, 76], n = 9. Для елемента з індексом i дочірні вузли мають індекси 2i + 1 та 2i + 2. Перший вузол, що має дочірні, має індекс ⌊n/2⌋ − 1 = 3, тому процедура Sink викликається для i = 3, 2, 1, 0.

- Крок 1 (i = 3, елемент 31). Дочірні елементи: 34 (індекс 7) та 76 (індекс 8). Більший із них 76, а 31 < 76, тому відбувається обмін. Масив: [19, 75, 43, 76, 5, 66, 62, 34, 31]. Елемент 31 опинився на індексі 8, дочірніх вузлів у нього немає, тому занурення завершується.
- Крок 2 (i = 2, елемент 43). Дочірні: 66 (індекс 5) та 62 (індекс 6). Більший 66 > 43, тому відбувається обмін. Масив: [19, 75, 66, 76, 5, 43, 62, 34, 31]. Елемент 43 на індексі 5 є листком.

- Крок 3 (i = 1, елемент 75). Дочірні: 76 (індекс 3) та 5 (індекс 4). Більший 76 > 75, тому відбувається обмін. Масив: [19, 76, 66, 75, 5, 43, 62, 34, 31]. Далі 75 порівнюється з дочірніми 34 та 31, 75 ≥ 34, тому елемент залишається на місці.
- Крок 4 (i = 0, елемент 19). Елемент проходить униз по трьох рівнях, і відбуваються три обміни. Спочатку 19 ↔ 76 (індекс 1), потім 19 ↔ 75 (індекс 3), потім 19 ↔ 34 (індекс 7). Для індексу 7 дочірні індекси 15 і 16 виходять за межі купи, тому занурення завершується.

Результат фази 1: [76, 75, 66, 34, 5, 43, 62, 19, 31]. Для кожного і виконується A[i] ≥ A[2i + 1] та A[i] ≥ A[2i + 2], тобто масив є максимальною купою, а найбільший елемент 76 стоїть у корені. На рисунку червоні лінії показують шлях занурення елемента, тобто позиції, між якими відбувалися обміни. У фазі 1 виконано 6 обмінів (1 + 1 + 1 + 3).

Пояснення до трасування та візуалізації фази 2 (сортування), рисунок 3.2

У цій фазі корінь купи (найбільший елемент) обмінюється з останнім елементом поточної купи й виходить із неї. Розмір купи зменшується на 1. Новий корінь порушує властивість купи, тому він заново занурюється викликом Sink(A, 0, n). Білі блоки на рисунку позначають елементи, що вже стали на свої остаточні місця.

- Крок 1: корінь 76 міняється місцями з останнім елементом 31, розмір купи зменшується до 8. Елемент 31 занурюється: 31 ↔ 75, 31 ↔ 34. Масив: [75, 34, 66, 31, 5, 43, 62, 19, 76].
- Крок 2: обмін 75 та 19, розмір купи 7. Занурення 19: 19 ↔ 66, 19 ↔ 62. Масив: [66, 34, 62, 31, 5, 43, 19, 75, 76].
- Крок 3: обмін 66 та 19, розмір купи 6. Занурення 19: 19 ↔ 62, 19 ↔ 43. Масив: [62, 34, 43, 31, 5, 19, 66, 75, 76].
- Крок 4: обмін 62 та 19, розмір купи 5. Занурення 19: 19 ↔ 43. Масив: [43, 34, 19, 31, 5, 62, 66, 75, 76].
- Крок 5: обмін 43 та 5, розмір купи 4. Занурення 5: 5 ↔ 34, 5 ↔ 31. Масив: [34, 31, 19, 5, 43, 62, 66, 75, 76].

- Крок 6: обмін 34 та 5, розмір купи 3. Занурення 5: 5 ↔ 31. Масив: [31, 5, 19, 34, 43, 62, 66, 75, 76].
- Крок 7: обмін 31 та 19, розмір купи 2. Обмінів при зануренні немає, оскільки 19 ≥ 5. Масив: [19, 5, 31, 34, 43, 62, 66, 75, 76].
- Крок 8: обмін 19 та 5, розмір купи 1. Купа складається з одного елемента, занурення не потрібне. Масив: [5, 19, 31, 34, 43, 62, 66, 75, 76].

Після восьми кроків (n − 1 = 8) отримано повністю відсортований масив [5, 19, 31, 34, 43, 62, 66, 75, 76]. На кроках 7 та 8 ліній на рисунку 3.2 немає, оскільки при зануренні обмінів не було. Загалом виконано 24 обміни (6 у фазі 1, а у фазі 2 – 8 обмінів кореня з останнім елементом та 10 обмінів при зануренні), що відповідає 24 · 3 = 72 присвоюванням при обмінах. З урахуванням службових присвоювань змінним k, j та n (за методикою ЛР1) загальна кількість присвоювань становить 141 (таблиця 3.3).

3. Аналіз нестійкості та обчислювальної складності алгоритмів сортування

3.1 Доведення нестійкості алгоритму HeapSort

Алгоритм сортування купою HeapSort належить до нестійких алгоритмів, оскільки під час обміну елементів може змінюватися початковий порядок елементів з однаковими ключами. У послідовності варіанта 22 повторюваних значень немає, тому для доведення розглянемо окремий масив із повторюваними значеннями, позначеними нижніми індексами, які відповідають їхнім початковим позиціям.

Розглянемо масив [3_1, 3_2, 1]: елементи 3_1 та 3_2 мають однаковий ключ, але різні початкові позиції. Для перевірки використано програму (Лістинг 3.3), у якій порівнюються лише ключі, а мітка зберігає початковий порядок однакових елементів. Стани масиву на кожному етапі наведено в таблиці 3.5.

Лістинг 3.3 – Python-код перевірки стійкості алгоритму сортування купою

```
def swap(arr, i, j):
```

```
    arr[i], arr[j] = arr[j], arr[i]

def sink(arr, i, n):
    """Занурення; порівнюються лише ключі (перший елемент пари)."""
    k = i
    while True:
        j = 2 * k + 1
        if j >= n:
            break
        if j + 1 < n and arr[j + 1][0] > arr[j][0]:
            j += 1
        if arr[k][0] >= arr[j][0]:
            break
        swap(arr, k, j)
        k = j

def heapsort(arr):
    n = len(arr)
    print("Початковий масив:", arr)
    for i in range(n // 2 - 1, -1, -1):
        sink(arr, i, n)
    print("Після побудови купи:", arr)
    for i in range(n - 1, 0, -1):
        swap(arr, 0, i)
        print("Обмін кореня з останнім:", arr)
        n -= 1
        sink(arr, 0, n)
        print("Після відновлення купи:  ", arr)
    return arr

# (ключ, мітка): мітка показує початковий порядок однакових ключів
data = [(3, '1'), (3, '2'), (1, '')]
print("Результат:", heapsort(data))
```

Таблиця 3.5 – Демонстрація нестійкості алгоритму HeapSort

Етап	Стан масиву
Початковий масив	[$З_1$, $З_2$, 1]
Побудова максимальної купи	[$З_1$, $З_2$, 1]
Переміщення кореня в кінець	[1, $З_2$, $З_1$]
Відновлення купи	[$З_2$, 1, $З_1$]
Переміщення кореня в кінець	[1, $З_2$, $З_1$]
Завершення сортування	[1, $З_2$, $З_1$]

У початковому масиві $З_1$ стоїть перед $З_2$, але після сортування їхній порядок змінився: $З_2$ стоїть перед $З_1$. Причина полягає в тому, що на другій фазі корінь купи обмінюється з останнім елементом, тобто елемент «перестрибує» через усі інші елементи, зокрема й через елементи з рівним ключем ($З_1$ потрапляє в кінець раніше за $З_2$).

Висновок: елементи з однаковими значеннями змінили свій початковий порядок, отже алгоритм HeapSort не забезпечує стійкості сортування.

3.2 Порівняльний аналіз алгоритмів HeapSort та QuickSort

Для порівняння ефективності алгоритмів HeapSort та QuickSort було підраховано кількість базових операцій для однакової послідовності варіанта 22 (n = 9, A = [19, 75, 43, 31, 5, 66, 62, 34, 76]). Використовувався єдиний підхід до підрахунку, аналогічний ЛР1: враховувалися порівняння значень елементів масиву та присвоювання, передбачені обраною методикою підрахунку, – як при обмінах (3 присвоювання на один обмін), так і службові присвоювання змінним, що керують ходом алгоритму (у HeapSort – k, j, n; у QuickSort – p, b). Для QuickSort використано схему розбиття Partition із лекції: опорний елемент вибирається функцією PickElement (першим елементом підмасиву) та обмінюється з останнім елементом A[h], після чого цикл розбиття виконується для i від l до h − 1 (опорний елемент сам із собою не порівнюється); обмін елемента із самим собою також виконується і враховується. Програмний код наведено в Лістингу 3.4, результати порівняння – в таблиці 3.6.

Лістинг 3.4 – Python-код швидкого сортування (QuickSort за схемою Partition із лекції) з підрахунком базових операцій

```
def partition(arr, l, h, cnt):
    """Розбиття за лекцією: PickElement - перший елемент підмасиву."""
    p = l  # p <- PickElement(A)
    cnt[1] += 1  # присвоєння p
    arr[p], arr[h] = arr[h], arr[p]  # Swap(A[p], A[h])
    cnt[1] += 3  # обмін = 3 присвоювання
    b = l  # b <- l
    cnt[1] += 1  # присвоєння b
    for i in range(l, h):  # позицію i = h займає опорний елемент
        cnt[0] += 1  # Compare(A[i], A[h])
        if arr[i] < arr[h]:
            arr[i], arr[b] = arr[b], arr[i]  # Swap(A[i], A[b])
            cnt[1] += 3  # обмін = 3 присвоювання
            b += 1  # b <- b + 1
            cnt[1] += 1  # присвоєння b
    arr[h], arr[b] = arr[b], arr[h]  # Swap(A[h], A[b])
    cnt[1] += 3  # обмін опорного елемента на своє місце
    return b

def quicksort(arr, l, h, cnt):
    """Швидке сортування за лекцією."""
    if l < h:
        p = partition(arr, l, h, cnt)  # p <- Partition(A, l, h)
```

```
        cnt[1] += 1  # присвоєння p
        quicksort(arr, l, p - 1, cnt)
        quicksort(arr, p + 1, h, cnt)

# Моделювання
A = [19, 75, 43, 31, 5, 66, 62, 34, 76]
cnt = [0, 0]  # [порівняння, присвоювання]
quicksort(A, 0, len(A) - 1, cnt)
print(f"Відсортований масив: {A}")
print(f"Загальна кількість порівнянь: {cnt[0]}")
print(f"Загальна кількість присвоювань: {cnt[1]}")
```

Під час сортування послідовності варіанта 22 функція Partition викликається 5 разів: для підмасивів [0..8], [2..8], [2..3], [5..8] та [7..8]. Кількість порівнянь становить 8 + 6 + 1 + 3 + 1 = 19. Кількість присвоювань складається з трьох частин: 5 викликів × 8 (p, обмін A[p] ↔ A[h] – 3, b, фінальний обмін A[h] ↔ A[b] – 3) = 40; 4 обміни в циклі розбиття × 4 (обмін – 3 та b ← b + 1) = 16; 5 присвоювань p ← Partition = 5; разом 40 + 16 + 5 = 61.

Таблиця 3.6 – Порівняння обчислювальної складності та кількості базових операцій алгоритмів сортування купою та швидкого сортування

Назва алгоритму	Теоретичні відомості: порівняння	Теоретичні відомості: присвоювання	Результати експерименту: порівняння	Результати експерименту: присвоювання
HeapSort	O(n log n)	O(n log n)	34	141
QuickSort	Середній: O(n log n); найгірший: $O(n^2)$	Середній: O(n log n); найгірший: $O(n^2)$	19	61

За результатами таблиці 3.6 алгоритм HeapSort виконав 34 порівняння та 141 присвоювання, тоді як QuickSort – 19 порівнянь та 61 присвоювання. Отже, для заданої послідовності швидке сортування потребувало менше і порівнянь, і присвоювань.

Обидва алгоритми мають середню часову складність O(n log n), проте HeapSort зберігає її навіть у найгіршому випадку, тоді як для QuickSort складність може зростати до $O(n^2)$. Побудова максимальної купи має складність O(n), оскільки більшість вузлів розташована на нижніх рівнях дерева й потребує невеликої кількості операцій занурення. На другій фазі виконується n − 1 = 8 кроків, на

кожному з яких процедура Sink має складність O(log n). Таким чином, загальна часова складність алгоритму HeapSort становить O(n log n).

Висновок: У ході лабораторної роботи № 3 було спроєктовано, реалізовано та протестовано алгоритм сортування купою мовою Python на послідовності варіанта 22 (n = 9). У результаті виконання алгоритму отримано відсортований масив [5, 19, 31, 34, 43, 62, 66, 75, 76]. Трасування та візуалізація підтвердили принцип роботи алгоритму: спочатку масив перетворюється на максимальну купу, а потім найбільший елемент послідовно переміщується в кінець масиву з відновленням властивостей купи. Експериментально досліджено порівняння та присвоювання під час виконання алгоритму й виконано порівняння із швидким сортуванням. Сортування купою має часову складність O(n log n) у найкращому, середньому та найгіршому випадках, не потребує додаткового масиву для сортування та належить до нестійких алгоритмів. Швидке сортування має середню часову складність O(n log n), проте в найгіршому випадку його складність становить $O(n^2)$. Отже, сортування купою забезпечує гарантовану часову складність O(n log n) незалежно від початкового порядку елементів. Нестійкість алгоритму доведено на прикладі масиву [3_1, 3_2, 1], після сортування якого порядок однакових елементів змінився на протилежний. За результатами підрахунку базових операцій для послідовності варіанта 22 HeapSort виконав 34 порівняння та 141 присвоювання, тоді як QuickSort – 19 порівняння та 61 присвоювань.

Посилання на GitHub: https://github.com/TserkovniyAlex/ta-lab3-22.git